UNITED STATES PATENT APPLICATION

Non-Provisional Application Under 35 U.S.C. § 111(a)

TITLE OF INVENTION

SYSTEM AND METHOD FOR AI-DRIVEN MULTIMODAL CONTENT GENERATION FOR MUSIC ARTISTS USING CONTENT DNA DERIVATION, ROTARY POSITION EMBEDDING TRANSFORMER LANGUAGE MODELS, AND SIMULATED GPU EXECUTION ENGINES

APPLICANT	B-Lawz Music LLC
ASSIGNEE	B-Lawz Music LLC
APPLICATION NUMBER	[To Be Assigned by USPTO]
FILING DATE	[To Be Filed]
ATTORNEY DOCKET NO.	[To Be Assigned]
U.S. CLASSIFICATION	G06N 3/04 · G06F 40/30 · G06T 13/00 · H04L 9/32
INTERNATIONAL CLASSIFICATION	G06N 3/04 · G06F 40/30 · G06T 13/00
PREPARED	May 4, 2026

NOTICE: This document is a patent application draft prepared for strategic and filing-preparation purposes. It has not been filed with the United States Patent and Trademark Office (USPTO) or any other patent authority. This document does not constitute legal advice. All claims, descriptions, and

drawings are subject to review and amendment by a registered patent attorney or agent prior to filing. Consult qualified patent counsel before submission.

CROSS-REFERENCE TO RELATED APPLICATIONS

This application claims priority to and incorporates by reference any and all provisional applications filed by the same inventor(s) related to AI-based content generation, transformer language model training, and simulated computational hardware for machine learning workloads.

FIELD OF THE INVENTION

The present invention relates generally to artificial intelligence systems for content creation, and more particularly to: (1) a decoder-only transformer language model employing rotary position embeddings (RoPE) and SwiGLU feed-forward networks for music-domain text generation; (2) a Content DNA derivation system that maps musical genre and tonal descriptors to a structured visual parameter space for deterministic video scene synthesis; (3) a multimodal content generation pipeline that orchestrates text, image, audio, and video generation workers through a planner-executor architecture; (4) a simulated GPU execution engine with instruction-level scheduling for platform-independent AI inference and training; and (5) a scoped, hash-based API key management system for securing access to AI generation endpoints.

BACKGROUND OF THE INVENTION

Music artists operating in the modern digital landscape are required to produce high-volume, platform-tailored content across channels including Instagram, TikTok, and YouTube. Existing AI content tools suffer from several limitations:

(a) General-purpose large language models lack domain-specific knowledge of music marketing vocabulary, platform conventions, and artist identity, leading to generic outputs that fail to reflect an artist's brand.

(b) Video generation systems typically rely on template-based approaches or require extensive prompting, producing output with low aesthetic coherence across scenes.

(c) Multimodal pipelines commonly require manual coordination between separate text, image, audio, and video generation tools, with no unified planner that derives cross-modality consistency from a single artist-defined input.

(d) Deployment of transformer inference on consumer or cloud hardware is constrained by the lack of portable, framework-independent execution layers capable of profiling and scheduling attention and matrix-multiply operations without requiring dedicated GPU hardware.

(e) AI API platforms commonly rely on opaque bearer-token authentication, lacking fine-grained scope enforcement, key rotation workflows, and usage telemetry tied to individual generation capabilities.

There exists a need for an end-to-end AI platform that addresses all of the above limitations in a unified, artist-centric system.

© 2026 B-Lawz Music LLC. All Rights Reserved.

SUMMARY OF THE INVENTION

The present invention provides a system and method designated herein as **MaxBooster** that comprises the following principal components:

1. A Decoder-Only Transformer Language Model ("MaxBooster LM") implementing:

- Rotary Position Embeddings (RoPE) computed via precomputed cosine/sine frequency tables applied to query and key projections;
- SwiGLU feed-forward networks with gated linear unit activations and two-thirds-dimensionality hidden layer sizing;
- Weight-tied input embedding and output projection layers;
- Causal self-attention with upper-triangular masking;
- Control token conditioning for platform, tone, goal, and scene-stage targeting during music content generation.

2. A Content DNA System that:

- Derives a seven-dimensional normalized parameter vector (energy, darkness, warmth, saturation, grain, complexity, seed) from musical genre and tonal input;
- Deterministically maps that vector to a color palette, background motion type, typography sizing, animation style, scene timing weights, vignette intensity, and film grain parameters;
- Applies per-scene micro-variation via a seed-offset function to produce aesthetic variety while maintaining cross-scene visual coherence.

3. A Video Generation Pipeline ("Video Studio") that:

- Accepts artist-defined inputs (idea, platform, genre, tone, goal, artist name, duration);
- Uses the MaxBooster LM with scene-stage control tokens to generate scene text for hook, body, build, drop, cta, and outro stages;

- Converts scene text and Content DNA into structured SceneConfig objects containing rendering parameters;
- Renders scenes in parallel via a thread-pool executor and composites them with DNA-derived transition types into a final MP4 output;
- Exposes an asynchronous job-polling API for non-blocking video rendering.

4. A Multimodal Content Generation Pipeline that:

- Accepts a single artist input and an intent specification;
- Executes a planner phase to produce a directed acyclic task graph of typed worker steps (text, image, audio, video);
- Executes workers in dependency order, passing prior outputs as inputs to downstream steps;
- Assembles a typed MultimodalPackage of assets keyed by platform slot (e.g., Instagram cover image, TikTok voiceover audio, YouTube short video).

5. A Simulated GPU Execution Engine comprising:

- A DigitalGPU subsystem implementing a VRAM allocator, a SIMD compute core with tiled GEMM, numerically-stable softmax, and causal batch attention operations, and an instruction scheduler with per-operation wall-time profiling;
- A HyperGPU backend abstraction providing flash-attention and mixed-precision linear layer primitives for higher-fidelity training simulation.

6. A Scoped API Key Management System that:

- Stores API keys as SHA-256 hashes with key prefixes, scope arrays, usage counters, and expiration timestamps in a relational database;
- Enforces per-request scope checks (read, write, train, generate, admin);
- Supports environment-variable-based key bypass for administrative operations without database round-trips;
- Provides key rotation and revocation endpoints with immediate invalidation semantics.

© 2026 B-Lawz Music LLC. All Rights Reserved.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a high-level system architecture diagram showing the relationship between the AI Training Server (Python FastAPI, port 9878), the API Server (Node.js/TypeScript Express, port 8080), and the AI Dashboard (React/Vite SPA) components.

FIG. 2 is a block diagram of the MaxBooster LM transformer decoder layer, showing the RoPE application to Q and K projections, SwiGLU feed-forward network, pre-normalization, causal masking, and weight-tied output projection.

FIG. 3 is a data flow diagram of the Content DNA derivation pipeline, tracing the path from musical genre and tone inputs through genre-to-DNA mapping, tonal delta application, seed computation, palette derivation, and final SceneConfig rendering parameter generation.

FIG. 4 is a sequence diagram of the Video Studio generation and rendering workflow, illustrating the path from user input through scene planning, DNA derivation, parallel scene rendering via ThreadPoolExecutor, compositing with DNA-derived transitions, and MP4 delivery via asynchronous job polling.

FIG. 5 is a directed acyclic graph (DAG) illustrating the Multimodal Content Generation Pipeline, showing the normalize, plan, and per-modality worker execution steps and their dependency ordering.

FIG. 6 is a component diagram of the DigitalGPU execution engine, including the VRAM allocator with handle-based array storage, the OpCode instruction set (GEMM, ADD, SOFTMAX, ATTENTION, GEMM_BIAS_RELU), the SIMDCore compute unit with tiled matrix operations, and the Scheduler with wall-time profiling.

FIG. 7 is a schema and data flow diagram of the API Key Management System, showing key creation with SHA-256 hashing, scope storage, per-request authentication and scope verification, usage counter increment, and key rotation and revocation flows.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

The following detailed description is presented to enable any person skilled in the art to make and use the invention. For purposes of explanation, specific details are set forth to provide a thorough understanding of the present invention. However, the invention may be practiced without these specific details. The descriptions of specific applications are provided only as representative examples.

I. System Overview

The MaxBooster system is implemented as a three-tier architecture:

Tier 1 — AI Training Server: A Python FastAPI application hosting the MaxBooster LM, DigitalGPU/HyperGPU engines, Video Studio rendering pipeline, Content DNA system, and API key database.

Tier 2 — API Server: A Node.js/TypeScript Express application acting as a reverse proxy, header propagation layer, and multimodal generation orchestrator for client-facing requests.

Tier 3 — AI Dashboard: A React/Vite single-page application providing the user interface for API key management, GPU cluster monitoring, training status inspection, and the Video Studio editing interface.

All three tiers communicate via authenticated REST APIs using the scoped API key system described in Section VII below.

© 2026 B-Lawz Music LLC. All Rights Reserved.

II. Decoder-Only Transformer Language Model (MaxBooster LM)

II.A. Architecture Overview. The MaxBooster LM is a decoder-only transformer of configurable dimensionality and depth, defaulting to: embedding dimension D=512, number of layers L=8, number of attention heads H=8, maximum sequence length T_{max}=1024. The model is parameterized by environment variables (AI_MODEL_DIM, AI_MODEL_LAYERS, AI_MODEL_HEADS, AI_MODEL_MAX_LEN) to support deployment at varying scales.

II.B. Rotary Position Embeddings (RoPE). Position information is injected via Rotary Position Embeddings rather than additive absolute or learned positional encodings. Frequency tables are precomputed as:

```
theta_i  = 1 / (10000 ^ (2i / D_head))    for i in [0, D_head/2)
freqs    = outer(t, theta)                 for t in [0, T_max)
rope_cos = cos(freqs),  rope_sin = sin(freqs)
```

These tables are stored as non-trainable buffers. During the forward pass, for each attention layer, the query and key tensors of shape [B, T, H, D_h] are transformed as:

```
[x1, x2] = split(x, dim=-1)
x_rope   = concat(x1*c - x2*s,  x1*s + x2*c, dim=-1)
```

where c and s are the cosine and sine tables sliced to the current sequence length T. This transformation encodes relative positional relationships directly into the dot-product attention scores, enabling the model to generalize across sequence lengths.

II.C. RoPE-Aware Causal Self-Attention. Each attention layer computes a fused QKV projection: `QKV = Linear(D, 3D, bias=False)(x)`. QKV is reshaped to [B, T, 3, H, D_h] and unbound to yield Q, K, V. RoPE is applied to Q and K independently. Attention logits are computed with scale factor $1/\sqrt{D_h}$ and a precomputed upper-triangular causal mask (set to $-\infty$ above the diagonal) is added to enforce autoregressive generation.

II.D. SwiGLU Feed-Forward Network. Each transformer layer's feed-forward sublayer implements a gated linear unit with Swish activation:

```
hidden_dim = round_up_to_multiple_of_64( D * expansion * 2/3 )
gate_proj  = Linear(D, hidden_dim * 2, bias=False)
[g, v]     = split(gate_proj(x), dim=-1)
output     = Linear(hidden_dim, D, bias=False)(silu(g) * v)
```

The two-thirds scaling of hidden dimensions reduces parameter count relative to standard four-times expansion FFNs while maintaining model capacity.

II.E. Weight Tying and Pre-Normalization. The output projection head shares weights with the input token embedding matrix, reducing parameter count and improving training stability. Each transformer sub-layer is preceded by LayerNorm (pre-normalization), with residual connections applied after each sub-layer.

II.F. Control Token Conditioning. The MaxBooster LM vocabulary is augmented with structured control tokens:

```
Platform: <PLATFORM_TIKTOK>, <PLATFORM_INSTAGRAM>, <PLATFORM_YOUTUBE>
Tone:     <TONE_ENERGETIC>, <TONE_EMOTIONAL>, <TONE_AGGRESSIVE>
Goal:     <GOAL_GROWTH>, <GOAL_ENGAGEMENT>, <GOAL_AWARENESS>
Stage:    <STAGE_HOOK>, <STAGE_DROP>, <STAGE_BODY>, <STAGE_CTA>
```

During inference, the applicable control tokens are prepended to the artist idea text, directing the model to generate stage-appropriate marketing copy in the artist's voice.

© 2026 B-Lawz Music LLC. All Rights Reserved.

III. Content DNA System

III.A. DNA Parameter Vector. The Content DNA system maps a tuple (idea, genre, tone) to a normalized seven-dimensional parameter vector:

```
DNA = {
  energy:     float [0,1]  -- kinetic intensity of visuals
  darkness:   float [0,1]  -- luminance depth
  warmth:     float [0,1]  -- color temperature axis
  saturation: float [0,1]  -- chromatic intensity
  grain:      float [0,1]  -- film grain texture amount
  complexity: float [0,1]  -- derived: energy*0.7 + saturation*0.3
  seed:       int          -- deterministic: int(MD5(idea)[:8], 16)
}
```

III.B. Genre-to-DNA Base Mapping. A lookup table (GENRE_DNA) defines base parameter vectors per music genre. Example: the trap genre maps to energy=0.90, darkness=0.88, warmth=0.25, saturation=0.80, grain=0.30. Tone adjustments (TONE_DELTA) define additive deltas applied on top of genre base values.

III.C. Palette Derivation. From the DNA vector, a six-color palette is derived:

```
base_hue    = 0.67 - (warmth * 0.55)
micro_shift = (seed % 31) / 360.0
bg1, bg2    = gradient endpoints from base_hue + saturation
accent      = complementary hue: (base_hue + 0.5) % 1.0
color_grade = categorical tag (vintage | neon | warm | cinematic)
```

III.D. Background Motion Type Selection. Background motion type is selected from a two-dimensional energy/darkness grid: HIGH energy + HIGH darkness → "plasma"; HIGH energy + LOW darkness → "wave"; LOW energy + HIGH darkness → "aurora"; LOW energy + LOW darkness → "radial" or "animated_gradient". Selection is randomized using the deterministic seed.

III.E. Per-Scene Micro-Variation. Each scene index *idx* receives a hue offset: `scene_seed = (dna.seed + idx × 17) % 360`, producing harmonically related color variation per scene.

IV. Video Generation Pipeline (Video Studio)

IV.A. Scene Text Generation. The VideoAgent constructs control-token-prefixed prompts and invokes the MaxBooster LM to generate scene text for each stage. Stage count is determined by duration thresholds, supporting short-form (hook/cta) and long-form (hook/build/drop/body/cta/outro) content.

IV.B. SceneConfig Construction. Scene text and Content DNA parameters are combined to produce a structured SceneConfig per scene, including background type, colors, texts, font size, y-position, animation type, fade timing, effects (including "breathing" at high energy), vignette intensity (0.35 + darkness × 0.30), and film grain amount (int(grain × 18)).

IV.C. Parallel Scene Rendering and Compositing. Scene clips are rendered in parallel using a ThreadPoolExecutor with at most three concurrent workers. DNA-derived transition type is applied during compositing: darkness > 0.70 → "fadeblack"; energy < 0.50 → "dissolve"; otherwise → "fade".

IV.D. Asynchronous Job Polling API. Video rendering is exposed as an asynchronous job. `POST /video/generate-ai` returns a *job_id*. `GET /video-job/{job_id}` returns status and the final MP4 URL once rendering completes.

V. Multimodal Content Generation Pipeline

V.A. Input Normalization. All multimodal requests pass through a normalization step that analyzes the input modality and extracts a canonical representation identifying musical attributes, language style, and platform constraints.

V.B. Task Planning. The normalized input is submitted to the MaxBooster LM in "planner" mode, which generates a validated directed acyclic task graph with typed worker steps. The plan specifies worker type, step type, and input sources.

V.C. Worker Execution Graph. Steps are executed in dependency order with typed workers (text, image, audio, video), passing prior outputs as inputs to downstream steps.

V.D. Pack and Slot Definitions. Hard-coded pack manifests define named campaign types with platform-specific slots such as: cover_image (Instagram), tt_voiceover_audio (TikTok), tt_short_video (TikTok), yt_description_text (YouTube).

© 2026 B-Lawz Music LLC. All Rights Reserved.

VI. Simulated GPU Execution Engine

VI.A. DigitalGPU Architecture. The DigitalGPU provides a platform-independent execution environment for AI operations without requiring physical GPU hardware. It comprises:

- **VRAM:** A handle-based allocator mapping integer handle IDs to NumPy arrays with associated shape, dtype, and size metadata.
- **SIMDCore:** A compute unit implementing tiled GEMM (with configurable tile dimensions tile_m, tile_n, tile_k), element-wise ADD, numerically-stable SOFTMAX (per-row maximum subtraction), and batched causal ATTENTION (scaled dot-product with optional upper-triangular mask).
- **OpCode Instruction Set:** GEMM, ADD, SOFTMAX, ATTENTION, GEMM_BIAS_RELU.
- **Scheduler:** Iterates an ordered program of typed instructions, dispatches to SIMDCore, writes results back to VRAM, and records per-instruction wall-clock execution time in a profiling record.

VI.B. HyperGPU Backend Abstraction. The HyperGPU provides a higher-level backend for more realistic mixed-precision training simulation, including a flash-attention primitive (configurable block_size=32), mixed-precision linear layers, and GELU-activated feed-forward networks. This dual-engine approach enables profiling of both instruction-level compute costs (DigitalGPU) and realistic training dynamics (HyperGPU) on standard CPU hardware.

VII. API Key Management System

VII.A. Key Structure and Storage. API keys are generated with a human-readable prefix ("mbs_") followed by a cryptographically random suffix. Keys are stored as SHA-256 hex digests (key_hash) with associated prefix, scope array (["read", "write", "train", "generate", "admin"]), is_active flag, request_count, and temporal metadata (created_at, last_used_at, expires_at). The raw key is returned to the user exactly once and never persisted.

VII.B. Authentication Flow. On each API request: (1) extract raw key from X-Api-Key or X-Admin-Key header; (2) if the raw key matches ADMIN_KEY or AI_TRAINING_KEY_PROD environment variables, return a synthetic full-scope record (env bypass); (3) otherwise, compute SHA-256(raw_key) and query for an active, non-expired record; (4) if found, increment request_count and update last_used_at; (5) enforce required scope via dependency injection.

VII.C. Key Lifecycle Operations. Create: generate random key, compute hash/prefix, insert record, return raw key once. Rotate: generate new key, update hash/prefix, preserve scopes/metadata, immediately invalidate prior key hash. Revoke: set is_active=FALSE, preserving usage telemetry.

VII.D. Header Propagation. The Node.js API Server propagates X-Api-Key and X-Admin-Key headers to the Python AI Training Server, ensuring scope enforcement occurs at the authoritative backend layer regardless of which tier the request enters.

© 2026 B-Lawz Music LLC. All Rights Reserved.

CLAIMS

What is claimed is:

1. A computer-implemented system for AI-driven multimodal content generation for music artists, comprising:

a) at least one processor;

b) at least one non-transitory computer-readable medium storing instructions that, when executed by the at least one processor, cause the system to:

i) receive an artist input comprising at least an idea, a musical genre, a tonal descriptor, and a target platform;

ii) derive a Content DNA parameter vector from the musical genre and tonal descriptor, wherein the Content DNA parameter vector comprises at least energy, darkness, warmth, saturation, grain, complexity, and seed dimensions;

iii) generate scene text using a decoder-only transformer language model conditioned on control tokens encoding the target platform, tonal descriptor, and scene stage;

iv) map the Content DNA parameter vector to rendering parameters comprising at least a color palette, a background motion type, scene timing weights, typography sizing, and film grain intensity;

v) render a plurality of video scenes in parallel using the generated scene text and the rendering parameters; and

vi) composite the rendered video scenes into a single video file using a transition type determined by the energy and darkness dimensions of the Content DNA parameter vector.

2. The system of claim 1, wherein the decoder-only transformer language model employs Rotary Position Embeddings (RoPE) applied to query and key projections of a multi-head self-attention mechanism via precomputed cosine and sine frequency tables derived from the formula theta_i = 1 / (base^(2i/D_head)).

3. The system of claim 2, wherein the decoder-only transformer language model further employs a SwiGLU feed-forward network wherein the hidden dimension is set to two-thirds of the product of the embedding dimension and an expansion factor, rounded up to the nearest multiple of sixty-four.

4. The system of claim 3, wherein an output projection matrix of the decoder-only transformer language model shares weights with an input token embedding matrix.

5. The system of claim 1, wherein deriving the Content DNA parameter vector further comprises:

a) looking up genre-specific base parameter values from a genre-to-DNA mapping;
b) applying additive tonal adjustment deltas from a tone-to-delta mapping;
c) computing a deterministic seed from a hash of the artist idea text; and
d) computing a complexity parameter as a weighted sum of the energy and saturation dimensions.

6. The system of claim 1, wherein mapping the Content DNA parameter vector to a background motion type comprises selecting a motion type from a plurality of candidate types based on a two-dimensional energy and darkness grid, wherein the selection is further randomized using the deterministic seed dimension of the Content DNA parameter vector.

7. The system of claim 1, wherein the rendering parameters further comprise per-scene micro-variation computed as a hue offset equal to the product of a scene index and a fixed offset constant modulo three hundred sixty, added to the deterministic seed dimension.

8. The system of claim 1, wherein parallel rendering of the plurality of video scenes is performed by a thread-pool executor with at most three concurrent rendering workers.

9. The system of claim 1, further comprising an asynchronous job management system wherein the rendering of the video file is initiated by a first API request returning a job identifier, and the completion status and output URL are retrievable via a second API request parameterized by the job identifier.

10. A computer-implemented system for multimodal content generation comprising a planner-executor pipeline, the system comprising:

a) a normalization module configured to receive an artist input and produce a canonical normalized representation;

b) a planning module configured to invoke a language model in planner mode to generate a directed task graph comprising typed worker steps selected from text generation, image generation, audio generation, and video generation;

c) an execution module configured to execute the typed worker steps in dependency order; and

d) an assembly module configured to collect typed asset outputs into a structured multimodal package keyed by platform-specific content slots.

© 2026 B-Lawz Music LLC. All Rights Reserved.

11. The system of claim 10, wherein the planning module validates the generated task graph against a schema requiring each step to specify a step type, a worker type, and an input source selected from normalized input or a prior step identifier.

12. The system of claim 10, wherein the platform-specific content slots are defined by a pack manifest mapping campaign types to ordered lists of platform slots, each slot specifying a modality, target platform, and content purpose.

13. A simulated GPU execution engine for platform-independent execution of artificial intelligence inference operations, comprising:

a) a virtual memory allocator (VRAM) configured to store numerical arrays indexed by integer handle identifiers with associated shape, data type, and byte size metadata;

b) a compute core configured to execute at least the following operations:

i) tiled general matrix multiplication (GEMM);

ii) element-wise tensor addition (ADD);

iii) numerically-stable softmax normalization (SOFTMAX); and

iv) batched causal self-attention (ATTENTION) comprising scaled dot-product computation with optional upper-triangular mask application;

c) an instruction scheduler configured to execute an ordered program of typed instructions, writing results back to the virtual memory allocator, and recording per-instruction wall-clock execution time in a profiling record.

14. The engine of claim 13, further comprising a higher-level backend abstraction layer implementing flash-attention operations with configurable block size and mixed-precision linear layer primitives, enabling simulation of training dynamics with reduced numerical precision.

15. A method for securing access to artificial intelligence generation endpoints using a scoped hash-based API key management system, the method comprising:

a) generating a cryptographically random API key comprising a human-readable prefix and a random suffix;

b) computing a cryptographic hash of the API key and storing the hash together with a key prefix, an array of authorized capability scopes, an active status flag, a usage counter, and temporal metadata in a relational database;

c) discarding the raw API key after returning it to the requesting party exactly once;

d) upon receipt of an API request, extracting a raw key from a request header, computing its cryptographic hash, and querying the database for a matching active, non-expired record;

e) upon a successful match, enforcing that the stored scope array contains the scope required by the requested endpoint;

f) incrementing the usage counter and recording the access timestamp; and

g) bypassing database lookup when the raw key matches a pre-authorized administrative key stored in a secure environment variable.

16. The method of claim 15, further comprising: rotating a key by generating a new random key and marking the prior key record as inactive in a single atomic operation; and revoking a key by setting its active status flag to false without deleting the key record, preserving usage telemetry.

17. The method of claim 15, wherein cryptographic hashing uses the SHA-256 algorithm, and wherein the key prefix stored in the database comprises the first twelve characters of the raw key.

18. The system of claim 1, wherein the control tokens conditioning the decoder-only transformer language model comprise at least one token selected from platform control tokens, tonal descriptor tokens, goal tokens, and scene-stage tokens, prepended to the artist idea text to form a conditioned generation prompt.

19. A non-transitory computer-readable medium storing instructions that, when executed by at least one processor, implement an end-to-end AI content generation platform for music artists, the instructions causing the processor to: (a) maintain a trained decoder-only transformer language model with rotary position embeddings, SwiGLU feed-forward sublayers, and control-token conditioning; (b) derive a Content DNA vector from musical genre and tonal descriptors; (c) generate platform-specific content across text, image, audio, and video modalities via a planner-directed worker pipeline; (d) render video content with DNA-derived parameters and transitions; (e) simulate GPU execution environments via instruction-scheduled virtual compute cores; and (f) authenticate all generation requests via a scoped, hash-based API key system with key rotation and revocation.

20. The medium of claim 19, wherein the platform further exposes a web-based dashboard enabling an artist or developer to manage API keys, monitor GPU cluster simulation status, inspect model training state, and operate the video generation studio through a browser-based interface without requiring installation of local software.

© 2026 B-Lawz Music LLC. All Rights Reserved.

ABSTRACT

An end-to-end artificial intelligence platform for music artist content generation comprising: (1) a decoder-only transformer language model implementing Rotary Position Embeddings (RoPE) and SwiGLU feed-forward networks with control-token conditioning for music-domain text generation; (2) a Content DNA system that deterministically derives a seven-dimensional visual parameter vector from musical genre and tonal inputs and maps it to color palettes, background motion types, scene timing weights, and rendering constants; (3) a video generation pipeline that generates stage-structured scene text, renders scenes in parallel, and composites them with DNA-derived transitions into MP4 outputs via an asynchronous job API; (4) a multimodal content generation pipeline employing a planner-executor architecture to coordinate text, image, audio, and video generation workers into platform-slot-keyed asset packages; (5) a simulated GPU execution engine with VRAM allocation, tiled GEMM, causal attention, and instruction-level profiling for platform-independent AI workload simulation; and (6) a scoped hash-based API key management system with SHA-256 storage, scope enforcement, usage telemetry, and key rotation and revocation primitives.

INVENTOR DECLARATION AND ASSIGNMENT

I hereby declare that:

(a) Each claim in this application is fully supported by the specification and drawings.

(b) I am the original and sole/joint inventor(s) of the subject matter claimed herein and for which a patent is sought.

(c) I acknowledge the duty to disclose information which is material to patentability as defined in 37 C.F.R. § 1.56.

(d) I hereby assign all right, title, and interest in and to this invention and the patent application to **B-Lawz Music LLC**, a limited liability company, its successors and assigns.

_____

Inventor Name (Print)

_____

Title / Role

_____

Signature

_____

Date (MM/DD/YYYY)

On Behalf of: B-Lawz Music LLC

_____

Registered Address

_____

City, State, ZIP

— END OF PATENT APPLICATION DRAFT — B-LAWZ MUSIC LLC © 2026 —

© 2026 B-Lawz Music LLC. All Rights Reserved.